



第三章作业

1. 以下关于用搜索算法求解最短路径问题的说法中，不正确的是（ ）。
 - A. 给定两个状态，可能不存在两个状态之间的路径；也可能存在两个状态之间的路径，但不存在最短路径（如考虑存在负值的回路情况）。
 - B. 假设状态数量有限，当所有单步代价都相同且大于 0 时，深度优先的图搜索是最优的。
 - C. 假设状态数量有限，当所有单步代价都相同且大于 0 时，广度优先的图搜索是最优的。
 - D. 图搜索算法通常比树搜索算法的时间效率更高。

B

2. 以下关于启发函数和评价函数的说法中正确的是（ ）。
 - A. 启发函数不会过高估计从当前节点到目标结点之间的实际代价。
 - B. 取值恒为 0 的启发函数必然是可容的。
 - C. 评价函数通常是对当前节点到目标节点距离的估计。
 - D. 如果启发函数满足可容性，那么在树搜索 A* 算法中节点的评价函数值按照扩展顺序单调非减；启发函数满足一致性时图搜索 A* 算法也满足该性质。

B

3. 假如可以对围棋的规则做出如下修改，其中哪个修改方案不影响使用本章介绍的 Minimax 算法求解该问题？（ ）
 - A. 由双方轮流落子，改为黑方连落两子后白方落一子。
 - B. 双方互相不知道对方落子的位置。
 - C. 由两人对弈改为三人对弈。
 - D. 终局时黑方所占的每目（即每个交叉点）计 1 分，且事先给定了白方在棋盘上每个位置取得一目所获取的分数，假设这些分数各不相同。双方都以取得最高得分为目标。

A

4. 下列关于探索与利用的说法中, 不正确的是 ()。
- A. 在多臂赌博机问题中, 过度探索会导致算法很少主动去选择比较好的摇臂。
 - B. 在多臂赌博机问题中, 过度利用可能导致算法对部分臂膀额奖励期望估计不准确。
 - C. 在 ϵ 贪心算法中, ϵ 的值越大, 表示算法越倾向于探索。
 - D. 在多臂赌博机问题中, 某时刻 UCB1 算法选择的臂膀置信上界为 R , 则此时任意摇动一个臂膀, 得到的硬币数量不会超过 R 。

D

5. 下列关于蒙特卡洛树搜索算法的说法中, 不正确的是 ()。
- A. 选择过程体现了探索与利用的平衡。
 - B. 算法进入扩展步骤时, 当前节点的所有子节点必然都未被扩展。
 - C. 模拟步骤采取的策略与选择步骤不一定要相同。
 - D. 反向传播只需要更新当前路径上已被扩展的节点。

B

6. 如图 1 所示，假设每个节点代表一个状态，节点之间的箭头表示状态转移关系，箭头旁的数字表示状态转移的代价。若使用以下搜索算法寻找从状态 A 到状态 I 的路径，请画出算法终止（找到第一条路径）时的搜索树，并在搜索树中标出节点的扩展顺序，以及找到的路径。若有多个节点拥有相同的扩展优先级，则优先扩展对应路径字典序较小的节点。
- 1) 基于树搜索的广度优先搜索。
 - 2) 基于图搜索的深度优先搜索。

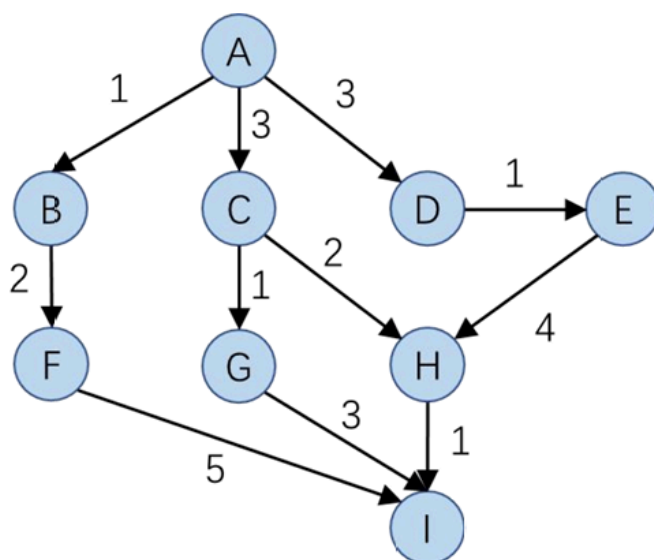
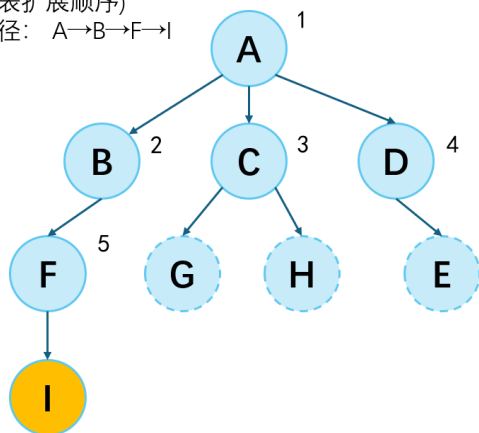
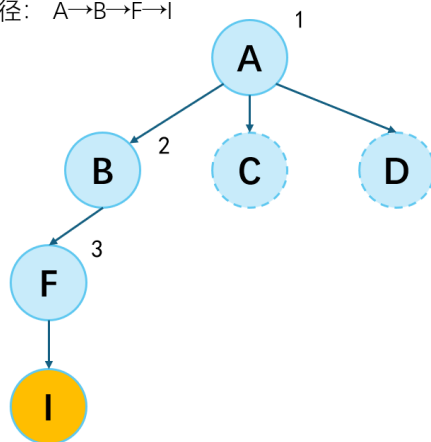


图 1 状态转移图

(1) BFS(优先扩展字典序小、即左边的节点)
 扩展顺序: A→B→C→D→F (节点右上数字
 代表扩展顺序)
 路径: A→B→F→I



(2) DFS(优先扩展字典序小、即左边的节点)
 扩展顺序: A→B→F
 路径: A→B→F→I



7. 考虑图 1 中的问题，给定每个状态的启发函数如表 1 所示。若仍以状态 A 为初始状态、状态 I 为终止状态，请分别使用以下算法求解从 A 到 I 的路径，并按照第 6 题中的方法画出搜索树。若有多个节点拥有相同的扩展优先级，则优先扩展对应路径字典序较小的节点。

表 1 启发函数的取值。

状态	A	B	C	D	E	F	G	H	I
启发函数	5	4	3	2	5	5	2	1	0

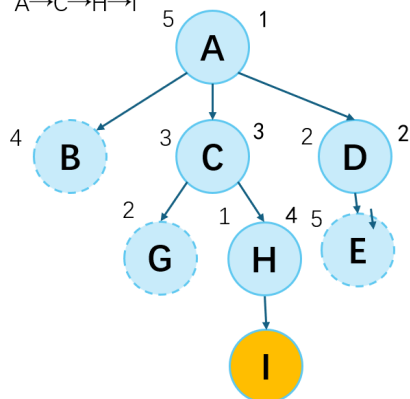
- (1) 基于树搜索的贪婪最佳优先搜索。
- (2) 基于图搜索的 A* 算法。

(1) 基于树搜索的贪婪最佳优先搜索

扩展顺序: A→D→C→H (节点右上数字代表

扩展顺序,左上数字代表评价函数的值)

路径: A→C→H→I

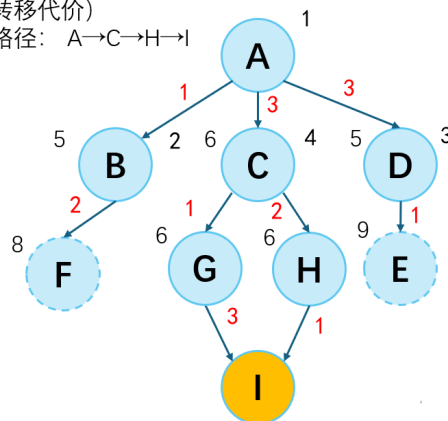


(2) 基于图搜索的 A* 算法

扩展顺序: A→B→D→C→G→H (箭头旁数字代表

转移代价)

路径: A→C→H→I



状态	A	B	C	D	E	F	G	H	I
h(n)	5	4	3	2	5	5	2	1	0

8. 根据问题 7 中的搜索过程，回答下列问题。

- (1) 贪婪最佳优先搜索和 A* 算法均能找到最短路径，且贪婪最佳优先搜索扩展的节点数更少，这是否与正文中的结论“A* 搜索是在已知信息下同类搜索策略中最优的”相矛盾，为什么？
- (2) 不难验证，问题 7 中给出的启发函数既是可容的也是一致的，从这个角度来说这是一个“好”的启发函数。显然这个启发函数并不足够“好”，以至于 A* 算法在该问题中效率低于贪婪最佳优先搜索。那么怎样的启发函数才能提高搜索效率？搜索效率最高时启发函数应该如何取值？

(1)

不矛盾。A* 搜索“在已知信息下同类搜索策略中最优”，指的是在保证最优性的搜索策略中，A* 能够最有效地利用启发信息，而不是说在每一个具体问题上扩展节点数都一定最少。贪婪最佳优先搜索只依据 $h(n)$ 选择节点，不考虑从初始状态到当前节点的实际代价 $g(n)$ ，因此一般不保证找到最短路径。问题 7 中贪婪最佳优先搜索也找到了最短路径且扩展节点较少，只是因为该启发函数在该例中恰好给出了较好的引导，不能说明贪婪搜索在一般意义上优于 A*。

(2)

能够提高搜索效率的启发函数应在满足可容性和一致性的前提下，尽可能接近真实剩余代价 $h^*(n)$ 。如果启发函数取值过小，虽然仍可能可容和一致，但提供的信息不足，A* 会扩展更多节点，效率较低。当搜索效率最高时，启发函数应取为： $h(n)=h^*(n)$ ，即每个节点到目标节点的真实最短代价。此时 A* 对剩余代价的估计完全准确，可以最大限度减少不必要的节点扩展。

9. 图 2 展示了一棵 Minimax 搜索树，可采用 alpha-beta 剪枝算法进行对抗搜索。假设对于每个节点的后继节点，算法按照从左向右的方向扩展。同时假设当 α 值等于 β 值时，算法不进行剪枝。请问：
- (1) 对图 2(a) 中所示搜索树进行搜索，请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的节点数量。
 - (2) 图 2(b) 展示了和图 2(a) 中完全相同的搜索树，只不过对搜索算法的顺序做了一些调整（扩展顺序交换的节点用红色虚线框标出）。请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的节点数量。

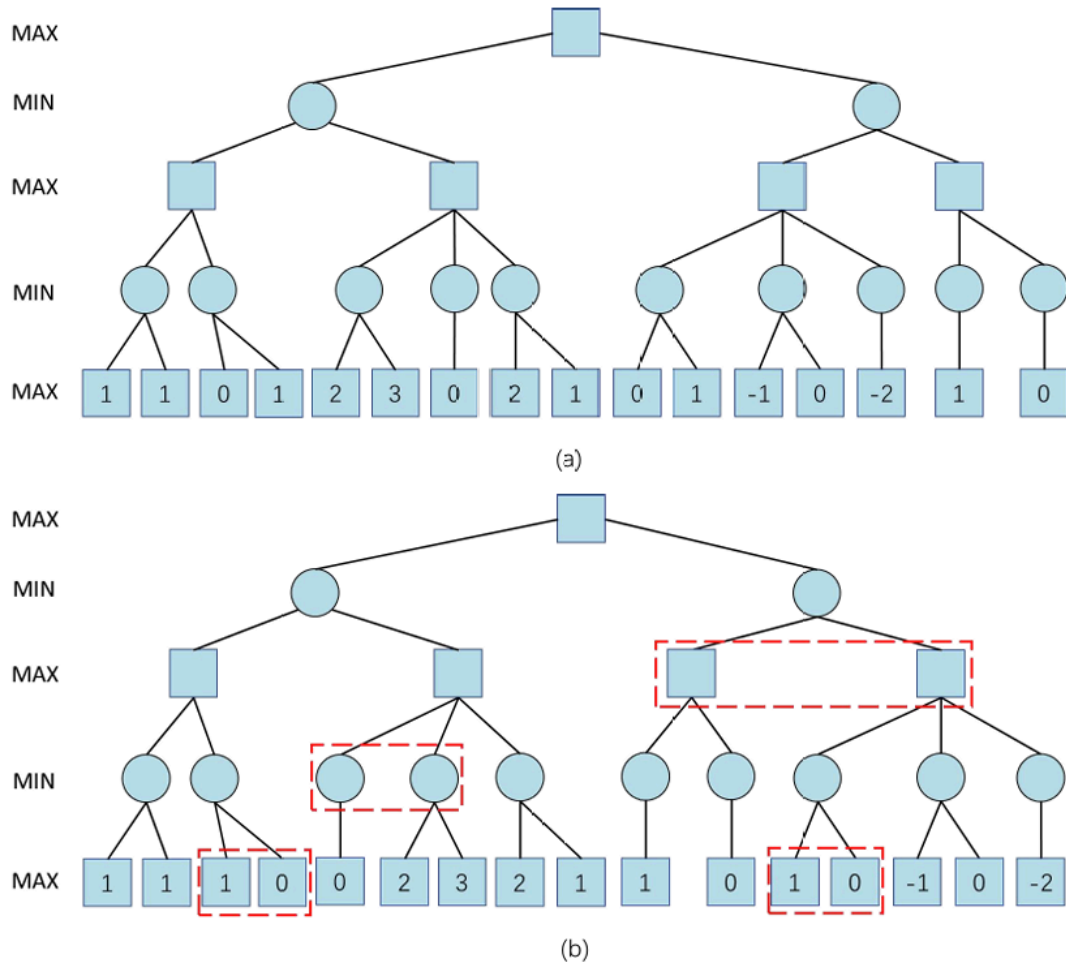


图 2 一棵 Minimax 搜索树。

[illegible]

MAX

MIN

MAX

MIN

MAX

10. 图 3 展示了一个蒙特卡洛树搜索的例子。其中每个叶子节点（终止节点）下标出了该节点对应的奖励。为了最大化取得的奖励，可利用蒙特卡洛树搜索求解奖励最大的路径。假设执行了若干步骤后，算法的状态如图 3 所示，节点内的数字分别表示“总奖励/访问次数”，虚线节点表示尚未扩展的节点。算法此时正要开始新一轮选择-扩展-模拟-反向传播的迭代

- (1) 假设 UCB1 算法中的超参数 $C = 1$ ，请计算并画出算法选择过程经过的路径。
- (2) 请继续执行扩展、模拟、反向传播步骤，并画出完成后的搜索树状态。（为了避免随机性，假设扩展总是扩展最左侧的未扩展节点，模拟总是选择最左侧的路径。）
- (3) 尝试进行若干次迭代，请问此时算法是否能有效地找到奖励最大的叶子结点（奖励为 9），那么进行足够多次迭代以后又如何？如果希望提高算法的效率，应该做出怎样的调整？

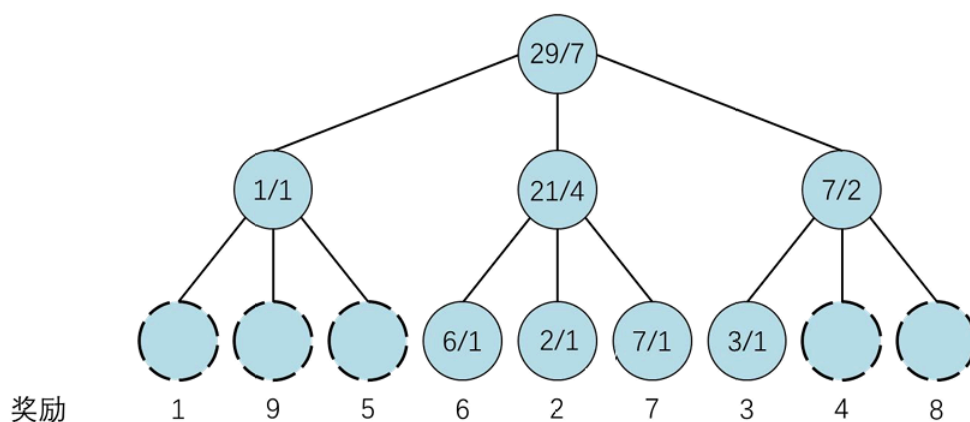
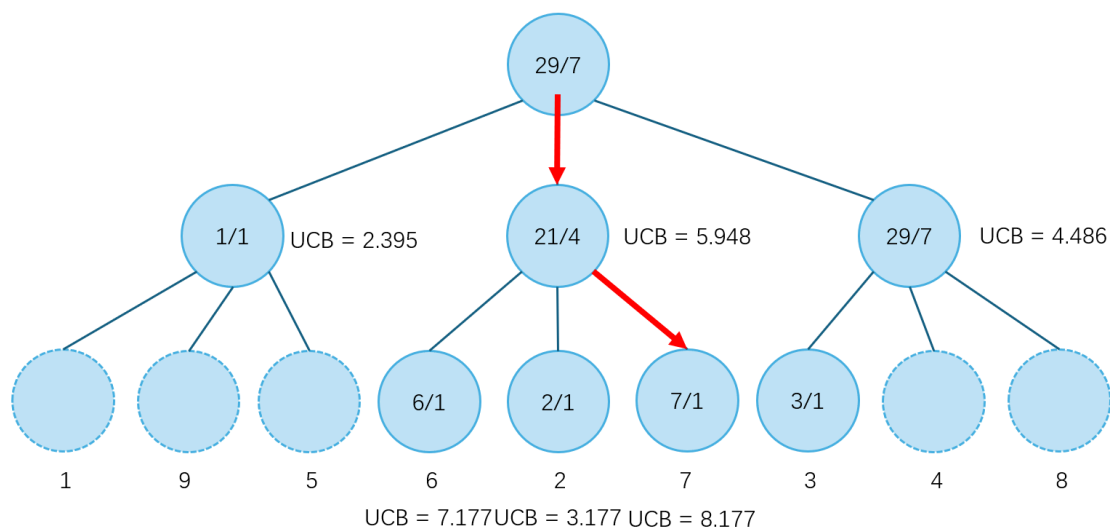
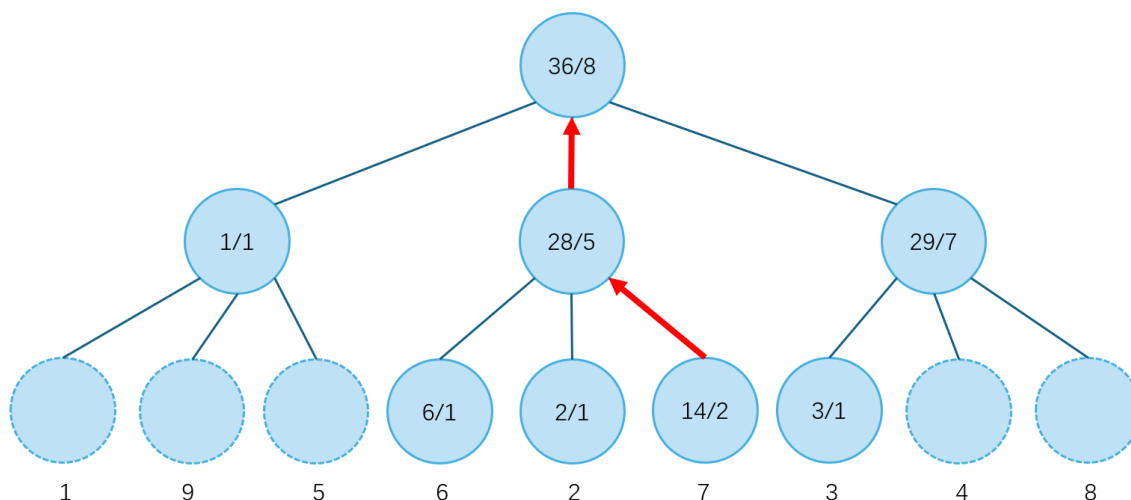


图 3 一棵蒙特卡洛树搜索的搜索树。

(1)



(2) 根据 UCB1 计算, 选择路径为根节点 $29/7 \rightarrow 21/4 \rightarrow 7/1$ 。由于 $7/1$ 已经是终止节点, 本轮无需扩展和模拟, 直接将奖励 7 反向传播, 得到根节点 $36/8$, 中间节点 $28/5$, 叶子节点 $14/2$ 。



(3) 短期内不一定能有效找到奖励最大的叶子 9, 因为左侧分支当前平均奖励 $1/1$ 较低, 算法更倾向于选择中间高平均奖励的分支。但足够多次迭代后, UCB 的探索项 $\sqrt{(\ln N / n)}$ 会促使算法重新探索访问次数少的分支, 从而有机会找到奖励 9。若希望提高效率, 可增大探索参数 c , 或改进模拟策略, 使算法更积极地探索未扩展节点。

11. 本章 3.2.4 小节中证明了在启发函数满足一致性时图搜索 A* 算法具有最优性, 证明过程中用到了一个没有详加说明的结论: 给定一条从 s_1 到 s_k 的最短路径

$s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_k$, 对于任意 $1 < i < k$ 和任意一条从 s_1 到 s_i 的最短路径 P ,

则路径 $P \rightarrow s_{i+1} \rightarrow \dots \rightarrow s_k$ 仍然是 s_1 到 s_k 的一条最短路径。试证明该结论。

设原路径 $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i \rightarrow s_{i+1} \rightarrow \dots \rightarrow s_k$ 为从 s_1 到 s_k 的一条最短路径。首先证明其前缀路径是从 s_1 到 s_i 的最短路径。

若不然, 则存在一条从 s_1 到 s_i 的路径 P' , 使得 $\text{cost}(P') < \text{cost}(s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i)$

于是用 P' 替换原路径中从 s_1 到 s_i 的部分，可得到一条从 s_1 到 s_k 的新路径 $P' \rightarrow s_{i+1} \rightarrow \dots \rightarrow s_k$

其总代价小于原路径总代价，这与原路径是从 s_1 到 s_k 的最短路径矛盾。

因此，原路径的前缀 $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i$ 必为从 s_1 到 s_i 的最短路径。

又因为 P 也是一条从 s_1 到 s_i 的最短路径，所以： $\text{cost}(P) = \text{cost}(s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i)$ ，于是路径 $P \rightarrow s_{i+1} \rightarrow \dots \rightarrow s_k$ 的总代价等于原路径 $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i \rightarrow s_{i+1} \rightarrow \dots \rightarrow s_k$ 的总代价。由于原路径是从 s_1 到 s_k 的最短路径，所以新路径 $P \rightarrow s_{i+1} \rightarrow \dots \rightarrow s_k$ 也必然是从 s_1 到 s_k 的一条最短路径。证毕。

12. 针对本章正文中算法 3.4.1 中给出的蒙特卡洛树搜索算法的伪代码，回答以下问题：

- (1) 函数 `BackPropagate` 中的第 3 行为何要减去而不是加上终局得分？如果要修改为加上终局得分，那么该得分应该以哪位玩家为视角得到的分数？
- (2) 算法 3.4.1 给出了利用蒙特卡洛树搜索算法求解对抗搜索问题的流程，如果要将蒙特卡洛树搜索应用于非对抗搜索，应该如何修改该算法？

(1) 算法 3.4.1 用于两人零和对抗搜索。在对抗搜索中，不同层节点对应的行动玩家交替变化，而双方收益相反。某一终局状态对一方有利，则对另一方不利。因此，反向传播时不能简单地把同一个终局得分加到所有路径节点上，而需要根据当前节点对应玩家的视角更新价值。`BackPropagate` 第 3 行使用减法，是为了体现父子节点之间玩家视角的转换。如果要改为加上终局得分，则该得分应当以当前节点对应玩家 `player(v.state)` 的视角计算得到的收益，即已经完成了玩家视角转换后的分数。

(2) 若将蒙特卡洛树搜索应用于非对抗搜索，则不存在玩家交替和双方收益相反的问题，所有节点都以最大化同一个奖励函数为目标。因此，反向传播时应直接累加本次模拟得到的奖励：

```
v.N ← v.N + 1
v.Q ← v.Q + reward(s_t)
```

而不再使用 `player(v.state)` 计算不同玩家视角下的效用，也不需要再在回传过程中改变收益符号。同时，选择步骤中的 UCB1 只需按照节点的平均奖励和探索项选择子节点，最终输出也应从“玩家 MAX 的最优动作”改为“当前状态下使期望奖励最大的动作”。

13. (*) 假设一棵 Minimax 搜索树的最大路径长度为 m ，每个非叶子结点恰好有 b ($b \geq 2$) 个孩子节点。若采用 alpha-beta 剪枝算法求解该问题，试证明在本章正文图 3.3.11 中所示最优情况下，算法扩展节点数量的上界为 $O(b^{\lfloor \frac{m+1}{2} \rfloor})$ 。

在普通 Minimax 搜索中，若搜索树最大路径长度为 m ，分支因子为 b ，则不剪枝时需要扩展的节点数为 $O(b^m)$ 。而在 alpha-beta 剪枝的最优情况下，节点扩展顺序最有利于剪枝，许多节点只需要考察一个子节点，就可以根据当前的 alpha、beta 值判断该分支不可能影响最终结果，从而剪去其余子树。

根据题设，除了每层最左端结点的孩子节点均被扩展外，其余节点均只扩展一个子节点。因此，搜索树的规模不是每下降一层都乘以 b ，而是大约每下降两层才乘以一次 b 。于是第 i 层被扩展节点数的数量级至多为 $O(b^{\lfloor (i+1)/2 \rfloor})$ 。

所以从第 0 层到第 m 层，总扩展节点数满足 $N(m) = O(1 + b + b + b^2 + b^2 + \dots + b^{\lfloor (m+1)/2 \rfloor})$ 。由于该和式是几何级数，其最高阶项为 $b^{\lfloor (m+1)/2 \rfloor}$ ，因此：
$$N(m) = O(b^{\lfloor (m+1)/2 \rfloor})$$

所以，在题设的最优扩展情况下，alpha-beta 剪枝算法扩展节点数量的上界为：
$$O(b^{\lfloor (m+1)/2 \rfloor})$$

14. (*) Dijkstra 算法是一个著名的求解最短路径的图搜索算法，该算法令每个节点评价函数的值为该节点对应路径的代价（即正文中定义的 $g(n)$ 函数）。

- (1) 当图中不存在负值代价时，Dijkstra 算法满足最优性。试根据本章中对图搜索 A* 算法最优性的讨论，证明该结论。
- (2) 假设我们已经通过其他方法证明了(1)中的结论，试根据该结论证明，启发函数满足一致性时，A* 算法满足最优性。

(1) Dijkstra 算法令每个节点的评价函数为从初始节点到该节点的路径代价 $g(n)$ 。而 A* 算法的评价函数为 $f(n) = g(n) + h(n)$ 。当取启发函数 $h(n) = 0$ 时，A* 的评价函数变为 $f(n) = g(n)$ ，此时 A* 算法退化为 Dijkstra 算法。

由于图中不存在负值代价，对于任意节点 n ，从 n 到目标节点的真实最短代价 $h^*(n)$ 非负，因此 $h(n) = 0 \leq h^*(n)$ ，即该启发函数是可容的。同时，对任意边 $n \rightarrow n'$ ，由于 $c(n, n') \geq 0$ ，有 $0 \leq c(n, n') + 0$ ，即 $h(n) \leq c(n, n') + h(n')$ ，所以该启发函数也是一致的。根

据图搜索 A* 在启发函数满足一致性时具有最优性的结论可知, Dijkstra 算法在不存在负值代价的图中满足最优性。

(2)

假设启发函数 h 满足一致性, 即对任意边 $n \rightarrow n'$, 有: $h(n) \leq c(n, n') + h(n')$

定义新的边代价: $c'(n, n') = c(n, n') + h(n') - h(n)$

由一致性可知: $c'(n, n') \geq 0$ 。因此, 在新边代价 c' 下, 图中不存在负值代价, 可以使用 Dijkstra 算法的最优性结论。

设从起点 s 到节点 n 的一条路径为: $s = n_0 \rightarrow n_1 \rightarrow \dots \rightarrow n_k = n$, 则该路径在新代价下的总代价为:

$$\begin{aligned} & \sum c'(n_i, n_{i+1}) \\ &= \sum [c(n_i, n_{i+1}) + h(n_{i+1}) - h(n_i)] \\ &= g(n) + h(n) - h(s) \end{aligned}$$

其中 $h(s)$ 是常数。因此, 在新图中按照 $g'(n)$ 最小选择节点, 等价于在原图中按照 $g(n) + h(n) = f(n)$ 最小选择节点。也就是说, 原图中的 A* 算法等价于在新图中运行 Dijkstra 算法。

由于新图边代价非负, Dijkstra 算法能找到新图中的最短路径。又因为对目标节点 t 有 $h(t) = 0$, 所以: $g'(t) = g(t) + h(t) - h(s) = g(t) - h(s)$, 其中 $h(s)$ 为常数, 最小化 $g'(t)$ 等价于最小化原图中的 $g(t)$ 。因此, A* 在原图中找到的路径也是从起点到目标节点的最短路径。故启发函数满足一致性时, A* 算法满足最优性。